Term Work

APB-Based SoC Peripheral Subsystem Integration and Verification

Phase 1: System Specification

Work Done By : Neethu Jaisan

AIM

The project aims to design and verify ***an AMBA APB-based SoC peripheral subsystem*** *integrating high-performance modules like* ***SRAM****, a* ***Timer****,* ***GPIO****, and an* ***ALU****. Using* ***SystemVerilog*** *for RTL and UVM for verification, the goal is to ensure seamless bus integration, address decoding, and protocol compliance through constrained-random testing.*

INTENDED APPLICATIONS

The SoC Peripheral Subsystem is designed as a low-power, modular controller for embedded systems.

INITIAL SPECIFICATION

The proposed SoC subsystem follows a **Master-Slave topology** centered around a dedicated **APB Interconnect (Address Decoder)**.

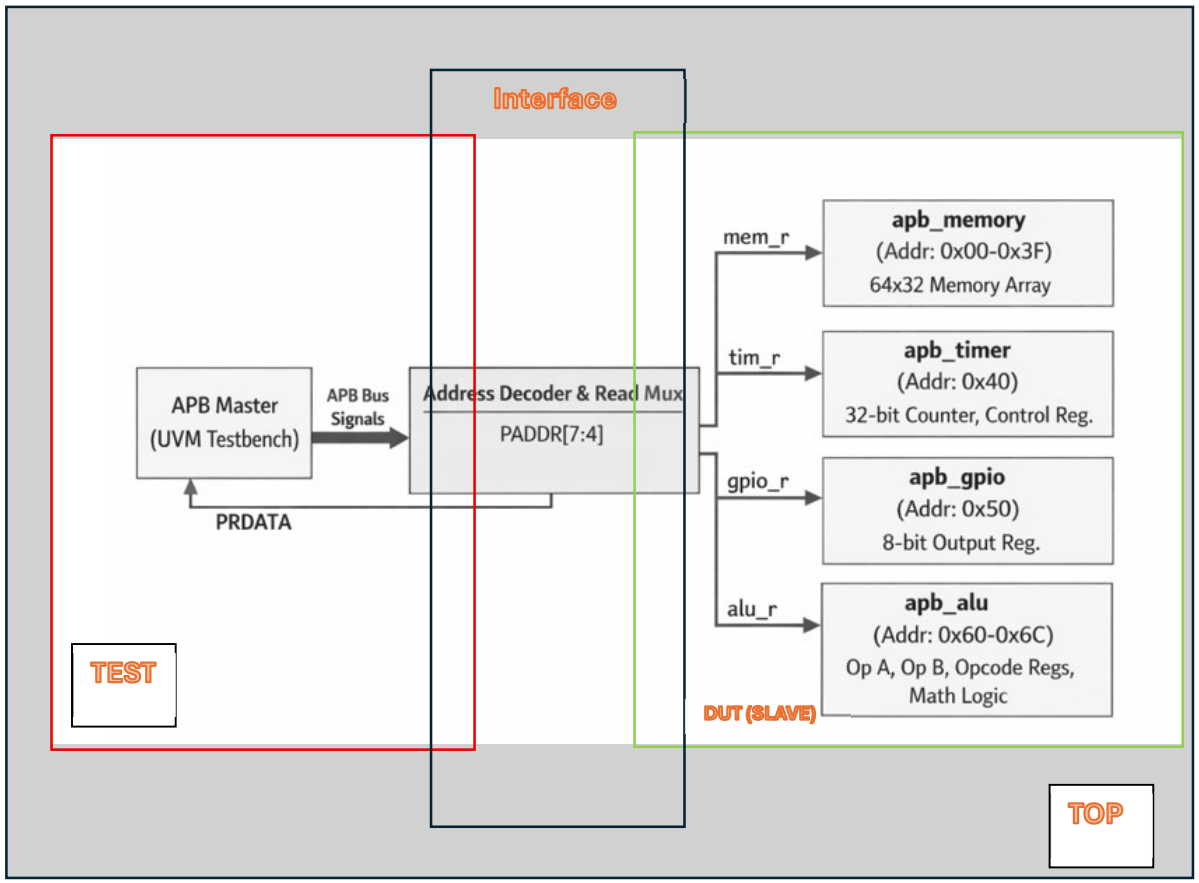

Fig 1: Block Diagram Featuring entire System

A. DUT Implementation Details

The Device Under Test (DUT) is an **APB-based SoC Peripheral Subsystem** that integrates four functional slaves through a central memory-mapped interconnect.

1. Slave Peripherals

- **Memory Block (apb_memory):** A 64-word SRAM for data storage. It supports synchronous single-cycle reads and writes.
- **Timer Module (apb_timer):** A 32-bit counter with a programmable enable register (at 0x40) and a read-only value register (at 0x44).
- **GPIO Block (apb_gpio):** An 8-bit output interface (at 0x50) that captures and holds bus data for external device control.
- **ALU Block (apb_alu):** A math co-processor with registers for Operands (A, B) and Opcode. It supports **Addition, Subtraction, AND, and OR** operations.

2. Implementation Features

- **Protocol:** Synchronous AMBA APB (IDLE, SETUP, ACCESS states).
- **Reset:** Asynchronous active-low reset (PRESETn) to initialize all registers.
- **Timing:** Writes are sampled on the PCLK rising edge; reads are combinational for low-latency feedback.

3. Address Map Specification

Peripheral	Base Address (Hex)	Accessible Range	Type
SRAM	0x00	0x00 - 0x3F	Read/Write
Timer	0x40	0x40 - 0x4F	Control/Status
GPIO	0x50	0x50 - 0x5F	I/O
ALU	0x60	0x60 - 0x6C	Computation

B. System Interface Details

The interface encapsulates the **APB** protocol signals, acting as the communication backbone between the Testbench and the DUT. The **Advanced Peripheral Bus (APB)** is a low-power, non-pipelined protocol within the **AMBA** family. It uses a simple two-phase handshake (**Setup** and **Access**) to connect **a master to low-bandwidth peripherals**.

- **Clock & Reset:** Provides the global **PCLK** for synchronization and **PRESETn** for asynchronous system initialization.
- **Bus Control:** Includes **PSEL** (Peripheral Select), **PENABLE** (Strobe signal), and **PWRITE** (Direction control) to manage the APB transfer phases.
- **Address & Data:** Features an 8-bit Address bus (**PADDR**) and dual 32-bit Data buses (**PWDATA** for writes, **PRDATA** for reads).
- **Encapsulation:** By grouping these signals into a single soc_if block, the design simplifies top-level connectivity and allows the UVM environment to access all bus signals via a single virtual interface handle.

Signal	Signal Name
PCLK	Clock signal
PRESETn	Reset signal
PADDR	32 bit address bus
PSELx	Select signal
PENABLE	Enable signal
PWRITE	Direction signal
PWDATA	32 bit Write Data bus
PREADY	Ready signal
PRDATA	32 bits read data bus

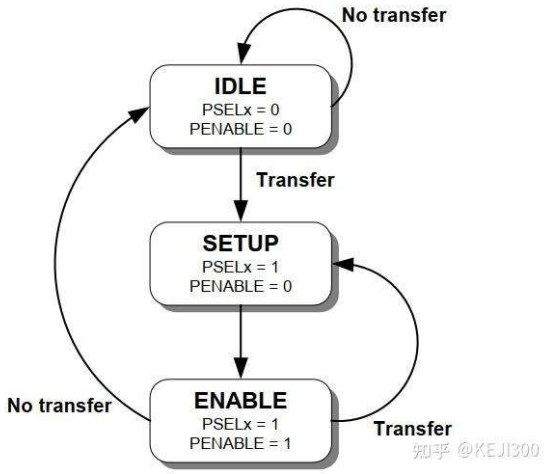

Fig:2 APB – Signal Definitions and State Diagram

IDLE: The default state. No data transfer is occurring; PSEL and PENABLE are both 0.

SETUP: When a transfer is required, the bus moves to the Setup state. PSEL is driven high, but PENABLE remains low. This state always lasts exactly one clock cycle.

ENABLE: The PENABLE signal is driven high. This is the "Access" phase where data is actually sampled or written.

- If no more transfers are needed, the bus returns to **IDLE**.
- If a new transfer is waiting, the bus can transition directly back to **SETUP**.

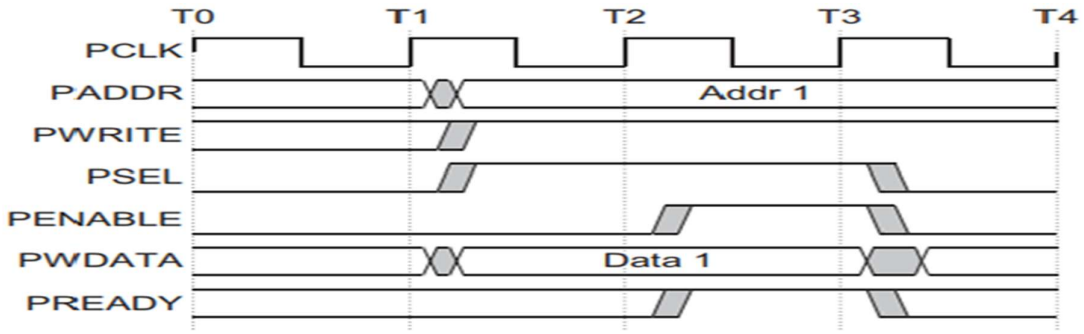

Fig:3 APB – Timing Diagram

Timing Diagram (Write Transaction)

- **T1 (Setup Phase):** The Master drives the Address (Addr 1), Write command (PWRITE), and Select signal (PSEL).
- **T2 (Access Phase begins):** The Master drives PENABLE high and provides the data (Data 1). Note that in this diagram, the slave keeps PREADY low, forcing the master to wait.
- **T3 (Completion):** The slave finally drives PREADY high. This indicates the slave is ready to capture the data.
- **T4:** On the rising edge of the clock, the transaction completes, and PSEL/PENABLE are dropped.

C. Testbench Implementation Plan

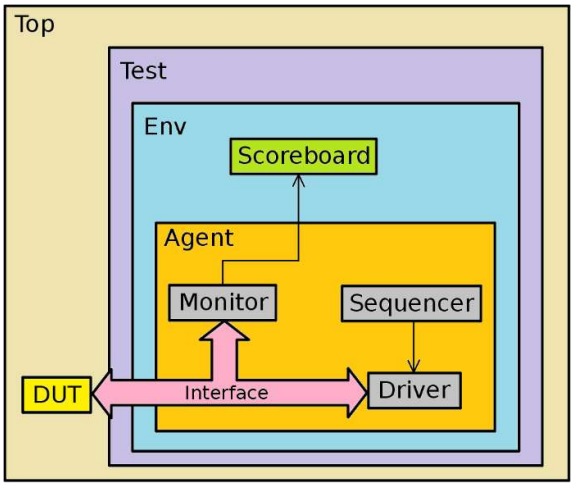

Fig:4 UVM -Testbench Architecture (General Model)

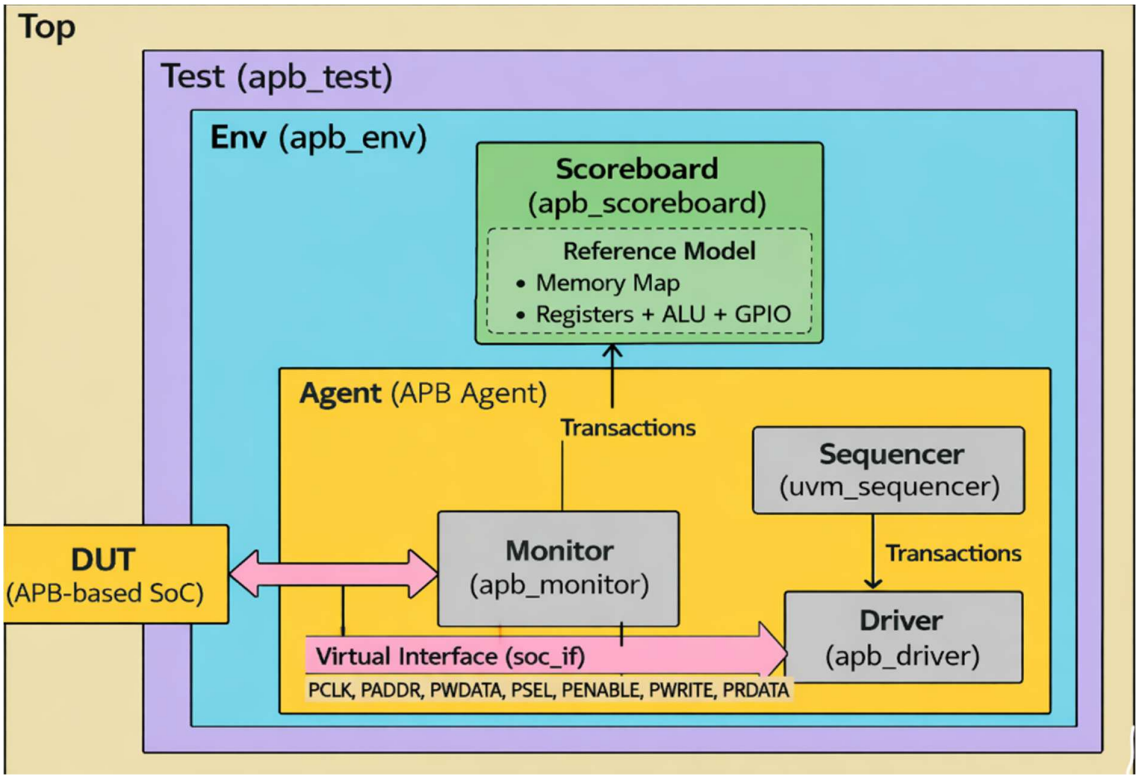

Fig:5 UVM -Testbench Architecture (for APB Based SoC Subsystem)

Component	Class Type	Specific Usage in SoC Project
Transaction	apb_txn	Encapsulates randomized PADDR[7:0], PWDATA[31:0], and PWRITE for bus activity.
Sequence	apb_seq	Generates specific "scenarios" (e.g., ALU Addition, Timer Reset, Memory Sweep).
Driver	apb_driver	Converts high-level transactions into **APB 3-State signals** (PSEL, PENABLE) on the soc_if.
Monitor	apb_monitor	Passively samples the soc_if to capture actual RTL behavior and sends it to the Scoreboard.
Sequencer	uvm_sequencer	Manages the flow of transactions between the Sequence and the Driver.
Scoreboard	apb_sb	Contains a **Reference Model** to predict ALU results and mirror SRAM data for validation.
Environment	apb_env	Top-level container that instantiates and connects all the above components.

Verification Feature Implementation Plan

Feature	Implementation Method	Purpose
Randomization	rand & constraints	To automatically generate diverse data patterns for the **ALU** and **SRAM**.
Assertions (SVA)	assert property	To monitor the soc_if for protocol violations (e.g., PENABLE rising without PSEL).
Functional Coverage	covergroup	To track if we have hit every register address and exercised all 4 ALU opcodes.
Reset Testing	PRESETn signaling	To verify that the **Timer** and **GPIO** registers return to 0 upon hardware reset.